# lab-08

AUTHOR
Andre Gala-Garza

PUBLISHED
March 12, 2026

**Disclaimer:** Generative AI was used to assist with templating and writing code in this assignment; however, this code was checked manually and edited by hand to ensure accuracy.

**Source:** OpenAI. (2026). *ChatGPT (GPT-5.4 Thinking)* [Large language model]. https://chatgpt.com/.

```
library(microbenchmark)
library(matrixStats)
library(parallel)
```

# 1. Vectorization

## Problem 1: Poisson distribution

```
fun1 <- function(n = 100, k = 4, lambda = 4) {
  x <- NULL

  for (i in 1:n){
    x <- rbind(x, rpois(k, lambda))
  }

  return(x)
}

fun1alt <- function(n = 100, k = 4, lambda = 4) {
  x <- matrix(0, nrow = n, ncol = k)

  x <- apply(x, 2, rpois, lambda = lambda)

  return(x)
}
```

```
poisson_1 <- fun1(100, 4, 4)
poisson_1alt <- fun1alt(100, 4, 4)
head(poisson_1)
```

```
     [,1] [,2] [,3] [,4]
[1,]    2    1    2    9
[2,]    9    4    2    4
[3,]    1    3    4    2
[4,]    4    5    4    1
```

```
[5,]    3    6    7    3
[6,]    1    5    0    4
```

```r
head(poisson_1alt)
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    1    1
[2,]    5    6    2    3
[3,]    1    4    5    2
[4,]    2    3    5    6
[5,]    4    6    7    6
[6,]    8    6    0    5
```

```r
dim(poisson_1)
```

```
[1] 100   4
```

```r
dim(poisson_1alt)
```

```
[1] 100   4
```

```r
# Benchmarking
microbenchmark(fun1(1000, 4, 4), fun1alt(1000, 4, 4))
```

```
Unit: microseconds
                expr     min       lq      mean   median       uq     max neval
   fun1(1000, 4, 4) 10749.1 13882.15 21375.849 17324.25 23779.80 63588.3   100
 fun1alt(1000, 4, 4)   438.1   560.50   783.173   643.25   838.65  5595.6   100
```

## Problem 2: Max value in each column

```r
# Data Generating Process (10 x 10,000 matrix)
set.seed(1234)
x <- matrix(rnorm(1e4), nrow=10)

# Find each column's max value
fun2 <- function(x) {
  apply(x, 2, max)
}

fun2alt <- function(x) {
  colMaxs(x)
}
```

```r
colMax_2 <- fun2(x)
colMax_2alt <- fun2alt(x)
```

```
head(colMax_2)
```

```
[1] 1.0844412 2.4158352 0.5747557 1.1022975 1.4494963 1.6478175
```

```
head(colMax_2alt)
```

```
[1] 1.0844412 2.4158352 0.5747557 1.1022975 1.4494963 1.6478175
```

```
length(colMax_2)
```

```
[1] 1000
```

```
length(colMax_2alt)
```

```
[1] 1000
```

```
# Benchmarking
microbenchmark(fun2(x), fun2alt(x))
```

```
Unit: microseconds
       expr    min      lq     mean median      uq    max neval
    fun2(x) 1566.2 1908.55 2611.029 2221.9 3116.75 7514.3   100
 fun2alt(x)   64.3   73.85  119.311   91.9  117.75 1912.0   100
```

# 2. Parallelization

## Problem 1: Serial bootstrap

```
my_boot <- function(dat, stat, R, ncpus = 1L) {

  # Getting the random indices
  n <- nrow(dat)
  idx <- matrix(sample.int(n, n*R, TRUE), nrow=n, ncol=R)

  # Make a cluster
  cl <- makeCluster(ncpus)
  on.exit(stopCluster(cl))

  clusterExport(cl, c("dat", "idx", "stat"), envir = environment())

  ans <- parLapply(
    cl = cl,
    X = seq_len(R),
    fun = function(i){
    stat(dat[idx[,i], , drop=FALSE])
```

```
  })

  # Converting the list into a matrix
  ans <- do.call(rbind, ans)

  return(ans)
}
```

## Problem 2: Parametric comparison

```
# Bootstrap of an OLS
my_stat <- function(d) coef(lm(y ~ x, data=d))

# DATA SIM
set.seed(1)
n <- 500; R <- 1e4

x <- cbind(rnorm(n)); y <- x*5 + rnorm(n)

# Checking if we get something similar as lm
ans0 <- confint(lm(y~x))
ans1 <- my_boot(dat = data.frame(x, y), my_stat, R = R, ncpus = 2L)

# You should get something like this
t(apply(ans1, 2, quantile, c(.025,.975)))
```

```
                  2.5%      97.5%
(Intercept) -0.1386903 0.04856752
x            4.8685162 5.04351239
```

```
##                   2.5%      97.5%
## (Intercept) -0.1372435 0.05074397
## x            4.8680977 5.04539763
ans0
```

```
                 2.5 %     97.5 %
(Intercept) -0.1379033 0.04797344
x            4.8650100 5.04883353
```

```
##                  2.5 %     97.5 %
## (Intercept) -0.1379033 0.04797344
## x            4.8650100 5.04883353
```

## Problem 3: Benchmarking

```r
# 1 core
system.time(
  my_boot(dat = data.frame(x, y), my_stat, R = R, ncpus = 1L)
)
```

```
   user  system elapsed
   0.54    0.06   14.76
```

```r
# 2 cores
system.time(
  my_boot(dat = data.frame(x, y), my_stat, R = R, ncpus = 2L)
)
```

```
   user  system elapsed
   0.43    0.25    8.06
```

From this benchmark, we find that using more than 1 core indeed takes less time to run the bootstrap than only using 1 core.

# 3. Short Answer

## Question 1

Parallelization is a good fit to solve complex problems that involve performing the same type of tasks repeatedly. The problems should also be independent of each other so that they can run at the same time while returning the same result.

## Question 2

Vectorization is a form of code optimization that is more focused on problem design and how the data is structured. Parallelization is more focused on using more efficient hardware to handle tasks that cannot be easily condensed into a single function, vector, or matrix. Because vectorization is simpler and more inherent to the problem implementation, it should ideally be considered first before resorting to parallelization, whose effectiveness will vary more depending on the hardware being used.